Les Modèles Algorithmiques Fondamentaux sur Collections de Données

Thème principal : Mécanismes d'accès et Modèles Algorithmiques Canoniques (Parcours complets et Recherche)

Synthèse exécutive : Ce cours formalise la démarche d'analyse préalable nécessaire avant toute rédaction de code en R1.01. L'informaticien doit d'abord identifier le mécanisme d'accès (Direct, Séquentiel ou Dynamique) de la collection de données traitée. Trois modèles algorithmiques canoniques sont structurés : le parcours complet systématique, le parcours complet conditionné et la recherche de première occurrence. La maîtrise des bornes d'indices (de 0 à N-1 pour éviter le plantage 'out of bounds'), l'application du pessimisme algorithmique et la mise en place d'une double condition d'arrêt pour la recherche sont obligatoires sous peine de fautes conceptuelles graves en évaluation.

Typologie & Mécanismes d'Accès aux Collections de Données

L'analyse algorithmique commence toujours par la qualification du mécanisme d'accès offert par la structure de données sous-jacente. Sur un tableau statique ou une chaîne de caractères (accès direct), l'adresse mémoire de la case d'indice 'i' est directement calculable par le matériel. En revanche, pour un fichier texte ou un flux CSV (accès séquentiel), le système d'exploitation n'expose que trois primitives : positionner le curseur au début, lire le composant suivant, et vérifier la fin de flux. Tenter de lire directement la 50ème ligne sans dérouler les 49 précédentes via une syntaxe T[50] est physiquement et conceptuellement impossible.

- Accès direct : Accès instantané à un élément par son indice T[i] en temps constant O(1).
- Accès séquentiel : Parcours obligatoire élément par élément depuis le début (début, suivant, fin).
- Accès dynamique : Structures hybrides proposant indexation et primitives séquentielles.
- Incompatibilité structurelle : Interdiction formelle d'utiliser l'indexation par indice T[i] sur un flux séquentiel.

Cas concret enseignant : *Le professeur met en garde contre l'erreur classique d'examen : rédiger une boucle 'Pour i allant de 0 à N' avec la notation 'T[i]' sur un fichier texte ou un fichier CSV brut. Comme le système d'exploitation ne propose qu'un curseur séquentiel avance-ligne, un tel algorithme est fondamentalement faux et entraîne la note zéro.*

Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique dès lors qu'un traitement uniforme et inconditionnel doit s'appliquer à l'ensemble des éléments d'une collection. En convention informatique standard (indexation à zéro), pour un tableau de taille N, la première case est à l'indice 0 et la dernière à l'indice N - 1. Le nombre d'itérations exécutées par la boucle bornée est exactement égal à (Fin - Début + 1), soit (N - 1 - 0 + 1) = N itérations.

- Exhaustivité : 100% des éléments de la collection sont traités.
- Boucle bornée Pour : Utilisation obligatoire car le nombre d'itérations est connu avant de démarrer.
- Gestion stricte des bornes : Pour N éléments indicés à partir de 0, l'indice varie de 0 à N - 1.
- Erreur Out of Bounds : Tentative de déréférencement d'une case hors limites mémoire (ex: T[N]).

Cas concret enseignant : *Exemple d'illustration vu en amphi : 'Afficher l'intégralité d'un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente chez les étudiants débutants consiste à écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat ('out of bounds') lors de la tentative de lecture de la case inexistante T[N].*

Second Modèle : Le Parcours Complet avec Traitement Conditionné

Ce modèle combine l'inspection exhaustive de la collection et l'application d'un filtre conditionnel. Chaque composant est examiné, mais le traitement n'est déclenché que si le composant valide une propriété logique. Lorsque la condition nécessite la comparaison de l'élément courant avec un élément voisin (par exemple T[i] >

T[i-1]), les bornes de la boucle doivent impérativement être décalées (démarrer à i = 1 au lieu de 0) sous peine de tenter d'évaluer T[-1] et de provoquer une faute critique d'accès mémoire.

- Parcours exhaustif : Inspection de l'intégralité des composants de la borne initiale à la borne finale.
- Traitement sélectif : Exécution de l'action conditionnée par le succès d'un prédicat booléen (Si ... Alors).
- Architecture algorithmique : Imbrication d'un test Si au sein d'une boucle Pour.
- Ajustement des bornes : Modification des limites de boucle en cas de comparaison contiguë (ex: T[i] et T[i-1]).

Cas concret enseignant : *Analogie de la chaîne de montage en usine donnée par le professeur : Chaque pièce défile systématiquement devant le contrôleur qualité (parcours complet d'inspection de toutes les pièces), mais le contrôleur n'appose le coup de tampon de rebut (action de traitement) que si et seulement si la pièce présente une fissure (traitement conditionné).*

Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux parcours complets, la recherche s'interrompt dès que le premier élément satisfaisant le critère est trouvé. Elle s'appuie sur le principe du pessimisme algorithmique. Le modèle exige DEUX conditions de sortie : trouver l'élément OU atteindre la fin de la collection. Deux variantes sont envisageables : la Variante 1 effectue une sortie immédiate en conservant la valeur exacte de l'indice i ; la Variante 2 utilise une boucle Tant que globale où l'incrémentation finale décale l'indice de +1, imposant une correction (i <- i - 1) si l'on souhaite exploiter la position exacte de l'élément.

- Pessimisme algorithmique : Initialisation de l'indicateur trouve à Faux au départ.
- Itération non bornée : Emploi obligatoire d'une boucle Tant Que (nombre d'itérations inconnu).
- Double condition d'arrêt : Succès (trouve = Vrai) OU Épuisement de la structure (fin de collection).
- Variante 1 (Rupture immédiate) : Sortie dès que trouvé, conserve l'indice exact de l'élément.
- Variante 2 (Tant que global) : Condition unique en tête, nécessite un réajustement i - 1 après la boucle.

Cas concret enseignant : *La règle d'or du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans la structure, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il rappelle qu'un algorithme de recherche présenté en examen avec une seule condition d'arrêt (oublier d'évaluer la fin de tableau) recevra systématiquement la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le modèle du parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, une structure dynamique autorisant à la fois l'accès direct et séquentiel peut être parcourue complètement. Cependant, il faut impérativement adapter le mode de itération au mode d'accès le plus efficient de la structure sous-jacente afin d'éviter une dégradation des performances.

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce qu'elles répondent à des besoins distincts : la Variante 1 (sortie anticipée/rupture) conserve l'indice exact sans correction post-boucle, ce qui est idéal si vous devez retourner un emplacement mémoire. La Variante 2 (boucle Tant Que globale) offre une structure épurée à test unique en tête, mais exige un réajustement (i - 1) à la sortie si l'indice exact est requis.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionné dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il faut réajuster la borne initiale de votre boucle bornée. On commence la boucle 'Pour' à i = 1 au lieu de i = 0. Cela garantit qu'au premier tour, T[i-1] évalue T[0], évitant ainsi l'accès illégal à T[-1] qui provoquerait une erreur 'Out of Bounds'.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur dans un tableau (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il faut impérativement utiliser le Modèle 2 (Parcours complet avec traitement conditionné) et non une recherche de première occurrence. Il est nécessaire d'examiner l'intégralité du tableau de 0 à N-1 pour traiter ou enregistrer chaque occurrence de la valeur recherchée.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments. Expliciter le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et choix du modèle. Il faut examiner tous les éléments d'un tableau (accès direct) de taille N et effectuer l'affichage uniquement pour les valeurs respectant la condition T[i] > 0. Le modèle approprié est le 'Parcours complet avec traitement conditionné'. Étape 2 : Détermination des bornes d'indices. Convention standard à zéro pour N éléments : première case = 0, dernière case = N - 1. Étape 3 : Choix de la structure itérative. Taille fixe et connue à l'avance -> Boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Écriture du filtre conditionnel. À chaque itération i, exécuter 'Si T[i] > 0 Alors Afficher(T[i]) Fin Si'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme réalise un parcours exhaustif sans risque de dépassement de mémoire (Out of Bounds) et applique le filtre sélectif de manière optimale.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on impérativement utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : Le Modèle 3 : Recherche de première occurrence, car le parcours doit s'interrompre dès que la valeur 10 est localisée.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce que les indices d'un tableau de N éléments commencent à 0. Les cases sont donc numérotées de 0 à N - 1. Accéder à l'indice N provoque une erreur de dépassement de capacité (Out of Bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence et quelles sont-elles ?

R : Elle doit obligatoirement comporter DEUX conditions d'arrêt : (1) Le succès de la recherche (trouve = Vrai) et (2) L'échec par épuisement de la collection (indice dépassant la borne de fin).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765